Build a Cubbie Skill — Hackathon Guide

Oct 19, 2026 · @ziyou

Cubbie, and what a Skill adds

Cubbie is an AI assistant that lives in a student's iMessage. It already knows the student's classes, can look things up, remember, remind, browse and make calls. A Skill teaches Cubbie one new student situation: when it matters, what to do, and which of Cubbie's abilities to use.

You do not build an app or a backend. You write a Skill in the same open format used by Claude and Codex, put it on GitHub, and any student can text the link to Cubbie to install it for themselves. Cubbie brings the abilities; your Skill brings the know-how.

What Cubbie can already do

Your Skill does not reimplement any of this. Name the ability you need in plain words ("check the student's Canvas due dates", "set a reminder for 8 pm") and Cubbie uses its own tool for it. Abilities marked *connected* work once the student has linked that account.

Ability	What Cubbie does	A Skill could...
Canvas (*connected*)	Reads courses, assignments and due dates, grades, announcements, discussions, course pages, files and Canvas inbox; searches and reads course materials	Turn a syllabus into a weekly study plan; warn before a grade-heavy week
Reminders and scheduled check-ins	Runs a task once or on a schedule and reaches the student by text or by **phone call**	Send a 9 pm "what's due tomorrow" check-in; call the student to wake them for an exam
Background tasks	Works on a longer job on its own and reports back when done	Compare ten apartment listings and send a shortlist later
Web	Searches the web, reads pages and PDFs, finds images	Pull today's dining hall menu or a club's event page
Browser	Opens a site in its own cloud browser, clicks and fills forms; the student can sign in or take over for a step such as payment	Book a study room on a campus site that has no API
Google (*connected*)	Gmail, Calendar, Drive, Docs, Sheets, Tasks and Contacts	Draft an email to a professor; add exam dates to the calendar
Email events (*connected*)	Searches the student's synced email and the events extracted from it	Find the internship interview time buried in an email thread

Ability	What Cubbie does	A Skill could...
Documents and slides	Creates Word, PDF, Excel, Markdown, calendar (.ics) and HTML files, and slide decks	Produce a one-page resume or a study guide PDF
Files and media	Reads photos, PDFs and documents the student sends; transcribes voice notes and videos	Turn a photo of a whiteboard into clean notes
Memory	Remembers facts and preferences about the student across conversations	Remember dietary needs or a budget and use them next time
Secure input	Sends a private page to collect a password, API key, phone or email; the value never enters the chat	Collect a key your Skill needs
Code sandbox	Runs Python and Node scripts in the student's private sandbox, with files that persist	Run your Skill's own scripts and command-line tools

Cubbie also writes replies, follows up proactively within the student's notification settings, and asks before it sends a message to someone else, submits work or changes an account.

What a Skill is

A Skill is a folder with a `SKILL.md` file: a short description of when to use it, then the instructions. It is the open Agent Skills format used by Claude, Codex, Cursor and many other agents, so a Skill you write for Cubbie works there too.

- New to Skills: the overview and the quickstart.
- Format reference: the specification.
- Writing well: Skill authoring best practices.

Cubbie adds only two things on top of the standard: name Cubbie's abilities in plain words in your instructions, and add a small `cubbie.yaml` when your scripts need packages, websites, keys or a login (other agents ignore it). A short example:

```
---
name: deadline-buddy
description: Keeps a student ahead of Canvas deadlines with a nightly check-in. Use
when the student wants help staying on top of assignments or reminders about due
dates.
---

1. Read the student's Canvas assignments due in the next 7 days.
2. Ask what time they want a nightly check-in; suggest 9 pm.
3. Create a daily scheduled check-in listing what is due in the next 48 hours.
```

Writing for Cubbie

On top of the general best practices, a few things matter most for a Skill that lives in a student's texts:

- **Start from one moment in a student's week.** "It's Sunday night and I have no idea what's due" beats "a deadline API".
- **Name abilities in plain words.** "Read the student's Canvas due dates", "create a daily check-in at 9 pm". Cubbie finds the exact parameters itself.
- **Be proactive with consent.** Recurring value comes from scheduled check-ins; let the student pick the time and how often.
- **Ask before acting for the student.** Sending, booking, submitting or paying always needs the student's yes; for payment, hand the browser to the student.
- **Write for a text thread.** Short messages, one idea each, links on their own line; put anything long in a document.
- **Plan for failure.** Say what to do when an account is not connected or nothing matches.
- **Keep personal data with the student.** Let Cubbie remember preferences; send data to your own server only when the Skill says so and the host is declared.

Adding scripts (optional)

Add scripts when your Skill needs something Cubbie cannot do on its own: call a specific service's API, run a calculation, parse a file, or use a command-line tool. Write the steps in `SKILL.md` as commands relative to your folder, for example `python3 scripts/search.py --major "Biology"`, and Cubbie runs them in the student's private sandbox.

When scripts need anything beyond the standard library, add a small `cubbie.yaml` next to `SKILL.md`:

```
cubbie: 1
dependencies:
  pip: ["requests==2.32.3"]          # exact versions
network:
  runtime: ["api.example.com"]       # hosts your scripts call
secrets:
  - env: EXAMPLE_API_KEY            # arrives as an environment variable
    label: Example API key
```

What your scripts can rely on:

- Linux with Python 3.12 and Node.js 22. Packages from `cubbie.yaml` are already on the path.
- `HOME` and `$SKILL_DATA_DIR` persist for each student, so logins and saved state survive. The Skill folder itself is a fresh copy each time the sandbox starts.
- HTTPS to the hosts you declared; everything else is blocked.
- Each run gets 2 minutes by default (up to 10), 1 CPU and 2 GB of memory. Print a short result; Cubbie reads the first 8,000 characters.

Scripts cannot call Cubbie's abilities directly. Have Cubbie fetch what it needs (Canvas, calendar, memory) first, then pass it to your script as arguments or a file. Declare packages in `cubbie.yaml`, never as install commands in `SKILL.md`: Cubbie installs them for you and ignores install steps in the text.

Sign-in and authorization

Pick the first approach on this list that works for your service. Every sign-in step a student has to do costs you installs, and Cubbie never asks for a password or key in the chat.

#	Approach	Use it when	What the student does	What you write
1	No sign-in	The data is public: menus, events, transit, weather, open APIs	Nothing	Nothing extra
2	Accounts already connected to Cubbie	You need Canvas or Google data	Nothing, or connect once when Cubbie asks	Name the ability in `SKILL.md`
3	Sign in on the website in Cubbie's browser	The service is a website with no API: campus portals, room booking, food ordering	Fills a secure login page once, or signs in by taking over the browser; takes over again for payment or two-factor codes	The site, the steps, and where the student must confirm
4	Command-line login with a link	You use a CLI that signs in through a browser link (device code style)	Taps the link and approves in their own browser	A `login` entry in `cubbie.yaml`
5	API key from the student	The service only issues personal API keys	Opens a secure page, pastes the key once, chooses Save	A `secrets` entry in `cubbie.yaml`, with a link to where the key is created

How they behave:

- **Saved for next time.** Logins from approaches 3 and 4 and keys from approach 5 stay with that student and are reused in later conversations. A key is matched by its variable name, so a student who already saved `OPENAI_API_KEY` is not asked again.
- **The CLI login must not wait.** The login command has to print the sign-in link and exit, or support a two-step form (start, then check). Commands that sit waiting for approval do not work yet.
- **Your team's own keys are not hosted.** Cubbie cannot keep your team's key and use it on behalf of every student. Use a backend that does not need a secret from the client, or ship a restricted, quota-limited key and assume anyone who installs the Skill can read it.
- **Never** ask the student to paste a password or key in the chat, hard-code a student's credentials, or send them to your own server.

Scenario playbooks

Starting points for the areas students care about most. The sign-in column refers to the numbered approaches above.

Scenario and example Skill	Cubbie abilities to combine	Sign-in	Watch out for
Food and dining. "Late-night food run": compare what is open, check the dining hall menu, prepare an order	Web, browser, memory (diet and budget), reminders	1 for menus; 3 for ordering sites	Cubbie never pays; the student takes over the browser to check out
Campus systems. "Study room booker": book a library room, check rec center slots or laundry	Browser, reminders, calendar	3	Sites differ by school; say which schools your Skill supports
Studying. "Exam sprint": turn the syllabus and course materials into a plan, practice questions and reminders	Canvas, documents, reminders and wake-up calls	2	Cite the course materials; help the student learn, do not do graded work for them
Money. "Budget buddy": track spending from receipts and email, find student discounts	Email events, files (receipt photos), memory, web	1 or 2	No bank logins and no moving money
Jobs and internships. "Internship radar": find postings that fit the student's major, track deadlines, draft cover letters	Web, scripts for a job board API, reminders, documents, Gmail drafts	1, or 5 if the API needs a key	Submit applications or send emails only after the student says yes
Friends and social. "Hangout planner": find a free evening, suggest places, set a reminder	Calendar, web, reminders	2	Cubbie texts only its own student; reaching friends goes through the student
Daily life. "Shuttle tracker": next campus shuttle, weather before class	Web, scripts for a public transit feed, reminders	1	Keep messages short and timely

Install, test and update

1. Push your Skill folder to a **public** GitHub repository.
2. Text Cubbie the link: the repository, a folder (`https://github.com/<you>/<repo>/tree/main/<skill-name>`) or the folder's `SKILL.md`.
3. Cubbie replies with a summary: what the Skill does, the exact commit, packages, websites, keys and logins. Reply yes to install. Nothing is installed before that reply.
4. If a key or login is needed, Cubbie sends the secure page or sign-in link next.
5. Talk to Cubbie the way a student would and check that your Skill triggers and finishes the job.

Each install is pinned to one commit. To ship a fix, push it and text Cubbie "update deadline-buddy"; Cubbie shows the new summary and asks again. "What Skills do I have?" lists them, and "remove deadline-buddy" uninstalls one. Installs are per student: a student who installs your Skill gets it only for themselves.

What is supported today

Supported	Not supported yet
Skills in the Agent Skills format, from public GitHub repositories	MCP servers, remote or local
Python and Node scripts; pinned pip and npm packages; prebuilt binaries by URL with a SHA-256	System packages (`apt`), install scripts (`curl | sh`), git or unpinned dependencies
HTTPS to the hosts you declare	Plain HTTP, SSH, databases, background servers
Sign-in through Cubbie's browser, CLI login links and student API keys	"Sign in with …" (OAuth) buttons for your own app; hosting your team's keys for all students
Cubbie's abilities, called by Cubbie as your `SKILL.md` describes	Scripts calling Cubbie's abilities directly; Cubbie texting people other than its student

Limits per Skill: at most 5 MiB and 300 files (1 MiB each), up to 10 hosts and 5 keys. Each student can install up to 20 Skills.

Submission and judging

Track 1 is complete when Cubbie can install your Skill from its GitHub link and run its main flow on a real account; it must be done within the first 10 hours. Submit the link to your Skill folder at a specific commit.

To be confirmed with the organizers:

- The submission form and the exact Track 1 cutoff time.
- Test accounts for teams, and whether submitted Skills are reviewed before students see them.
- Video guidelines (platforms, required tag, length) and how 7-day reach and votes are counted.

Appendix: cubbie.yaml reference

Omit any section you do not need; a Skill with only `SKILL.md` needs no `cubbie.yaml`. Unknown fields are rejected so a typo never silently does nothing.

Field	Meaning	Rules
`cubbie`	Format version	Must be `1`
`dependencies.pip`	Python packages	Exact versions only, `name==1.2.3`

Field	Meaning	Rules
`dependencies.npm`	npm packages, including CLIs	Exact versions only, `name@1.2.3`
`dependencies.download`	A prebuilt binary or archive	`url` (https), `sha256`, and `bin`: the executable to put on the path
`network.install`	Extra hosts needed only while installing	PyPI, npm and GitHub downloads are already allowed
`network.runtime`	Hosts your scripts call	Host names only; `*.example.com` allowed; no IPs; at most 10
`secrets`	Keys the student provides	`env` (uppercase letters, digits, _), `label`, optional `help_url`; at most 5; names such as `PATH`, `HOME` or starting with `CUBBIE_` or `SKILL_` are reserved
`login`	A CLI sign-in Cubbie starts for the student	`id`, `command` (prints a link, does not wait), optional `status` (exits 0 when signed in)

A fuller example:

```
cubbie: 1
dependencies:
  npm: ["netlify-cli@27.12.1"]
  download:
    - url:
https://github.com/example/tool/releases/download/v1.2.0/tool-linux-amd64.tar.gz
      sha256: "<64 hex characters>"
      bin: tool
network:
  runtime: ["api.netlify.com"]
login:
  - id: netlify
    command: netlify login --request "Cubbie wants to publish web pages for you"
--json
    status: python3 scripts/publish.py status
```